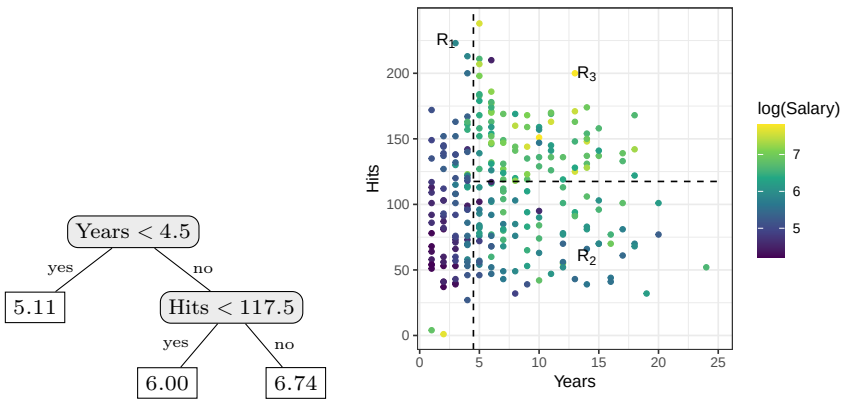
1 Decision Trees: Basic Idea

Tree-Based Methods: Overview

- Tree-based methods **stratify** (segment) the predictor space into a number of simple regions
- A new observation is predicted using the training observations in the region it belongs to: the **mean** response (regression) or the **most common class** (classification)
- The set of splitting rules can be summarised in a tree, hence the name **decision trees**
- A single tree is easy to interpret but is usually a weak predictor
- **Bagging**, **random forests** and **boosting** combine many trees, trading interpretability for accuracy

2 Regression Trees

Regression Trees: Example



- Response log(Salary); predictors `Years` in the major leagues and `Hits` in the previous year; each leaf shows the mean response in its region
- Years is the most important factor; Hits matters only for experienced players

Regression Trees: Prediction and Objective

- Divide the predictor space into J distinct, non-overlapping regions $R_1, \dots, R_J$
- Every observation falling in R_m receives the same prediction, the mean response of the training observations in R_m

$$\hat{y}_{R_m} = \frac{1}{|R_m|} \sum_{i:\, \mathbf{x}^{(i)} \in R_m} y^{(i)}, \qquad \hat{f}(\mathbf{x}) = \sum_{m=1}^{J} \hat{y}_{R_m} \, \mathrm{I}(\mathbf{x} \in R_m)$$

- Regions are chosen as high-dimensional **boxes** (rectangles) for simplicity and interpretability
- Goal: find boxes that minimise the residual sum of squares

$$\mathrm{RSS} = \sum_{m=1}^{J} \sum_{i:\, \mathbf{x}^{(i)} \in R_m} \left(y^{(i)} - \hat{y}_{R_m} \right)^2$$

Recursive Binary Splitting

- Considering every possible partition into J boxes is infeasible; use a **top-down, greedy** approach
- For predictor j and cutpoint s, define the pair of half-planes

$$R_1(j,s) = \{\mathbf{x} \mid x_j < s\} \quad \text{and} \quad R_2(j,s) = \{\mathbf{x} \mid x_j \geq s\}$$

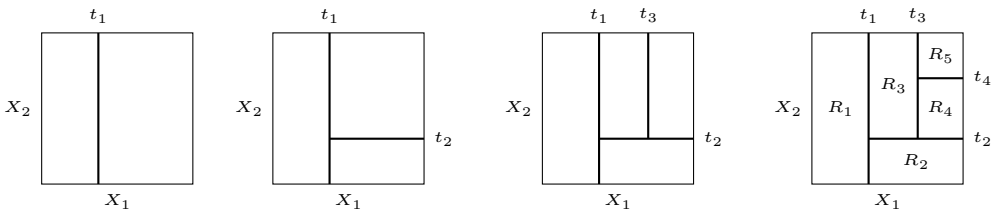
- Choose j and s that minimise

$$\sum_{i:\,\mathbf{x}^{(i)} \in R_1(j,s)} \left(y^{(i)} - \hat{y}_{R_1}\right)^2 + \sum_{i:\,\mathbf{x}^{(i)} \in R_2(j,s)} \left(y^{(i)} - \hat{y}_{R_2}\right)^2$$

Recursive Binary Splitting: Greedy Search and Stopping

- **Greedy**: the best split at the current step is chosen, even if a mediocre split now would enable a better one later
- After the first split, repeat the search inside each of the two resulting regions: choose the region, predictor j and cutpoint s that reduce the RSS the most
- Continue until a stopping criterion is met, e.g. no region has more than five observations
- The final regions $R_1, \ldots, R_J$ are the leaves of the tree, and a test observation is predicted by the mean response of the region it falls in

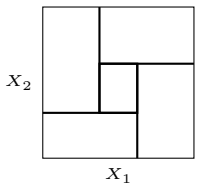
Building the Partition Step by Step



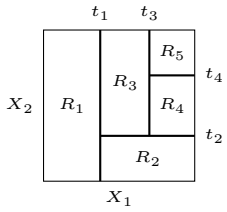
1st split 2nd split 3rd split 4th split

- The first split divides the whole space in two by a line perpendicular to one predictor axis
- Each later split divides *one* existing region, never the whole space
- Every split is a straight line parallel to a coordinate axis, so the final regions are boxes
- Each split is chosen to reduce RSS the most, given the splits made so far

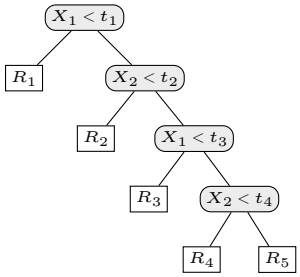
Partition, Tree and Invalid Partitions



Cannot result from recursive binary splitting



Can result from recursive binary splitting



The tree for the middle partition

- The left partition has no first split that cuts the whole space cleanly into two unions of its regions
- The tree is read top to bottom: left branch if the condition is true, right branch otherwise

Overfitting and Tree Pruning

- The resulting large tree fits the training data very well but overfits: high variance, poor test performance
- A smaller tree with fewer splits has lower variance and better interpretability at the cost of a little bias
- Stopping when the RSS decrease from a split is small is short-sighted, since a poor split may be followed by a very good one
- Better: grow a large tree T_0, then **prune** it back to obtain a subtree

Cost Complexity Pruning

- For each tuning parameter $\alpha \geq 0$, there is a subtree $T \subseteq T_0$ that minimises

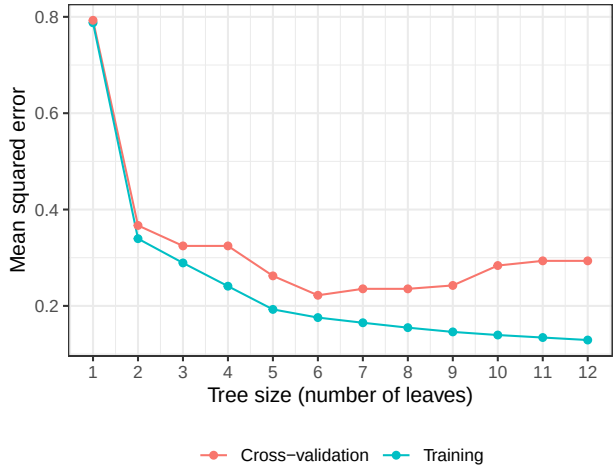
$$\sum_{m=1}^{|T|} \sum_{i:\, \mathbf{x}^{(i)} \in R_m} \left(y^{(i)} - \hat{y}_{R_m}\right)^2 + \alpha\, |T|$$

- $|T|$ is the number of terminal nodes (leaves) of T, and R_m is the region of the m-th leaf
- α controls the trade-off between fit to the training data and tree complexity
- $\alpha = 0$ gives T_0; as α increases, branches are pruned in a nested and predictable way, giving the whole sequence of subtrees
- Similar in spirit to the lasso, which penalises the size of the coefficients

Building a Regression Tree

1. Use recursive binary splitting to grow a large tree on the training data, stopping when each leaf has fewer than some minimum number of observations
2. Apply cost complexity pruning to obtain the sequence of best subtrees as a function of α
3. Use K-fold cross-validation to choose α: for each fold, repeat steps 1 and 2 on the other $K-1$ folds and evaluate the MSE on the held-out fold as a function of α; average over folds and pick the α with the smallest CV error
4. Return the subtree from step 2 that corresponds to the chosen α

Regression Tree: Choosing the Tree Size



- Hitters data with nine predictors, trees of increasing size
- Training error decreases monotonically with the number of leaves
- CV error is U-shaped; it picks a small tree
- The selected tree is simple and interpretable

Hyperparameters and Tuning: Background

- **Parameters** are learned from the training data, e.g. the splits and leaf means of a tree
- **Hyperparameters** are set *before* fitting and control model flexibility, e.g. the pruning parameter α, the number of trees B, the number of predictors m per split, the shrinkage λ and the depth d
- Training error always favours the most flexible setting (e.g. $\alpha = 0$ gives the largest tree), so hyperparameters must be chosen using an estimate of the **test** error
- The test set must not be used for this, otherwise it no longer gives an honest estimate of the final model's performance

Hyperparameter Tuning: Procedure

1. Choose a grid Γ of candidate values; with several hyperparameters, use all combinations (grid search) or a random sample of them (random search)
2. For each candidate $\boldsymbol{\gamma} \in \Gamma$, estimate the test error using a validation set or K-fold cross-validation
3. Choose the candidate with the smallest estimated error
4. Refit the model on the whole training set with the chosen $\hat{\boldsymbol{\gamma}}$, and evaluate it once on the test set

$$\hat{\boldsymbol{\gamma}} = \arg\min_{\boldsymbol{\gamma} \in \Gamma} \mathrm{CV}_{(K)}(\boldsymbol{\gamma}), \qquad \mathrm{CV}_{(K)}(\boldsymbol{\gamma}) = \frac{1}{K} \sum_{k=1}^{K} \mathrm{Err}_k(\boldsymbol{\gamma})$$

- $\mathrm{Err}_k(\boldsymbol{\gamma})$ is the error on fold k of a model fit with $\boldsymbol{\gamma}$ on the other $K-1$ folds
- The cost grows with the number of candidates times K model fits

3 Classification Trees

Classification Trees

- Same as a regression tree, but the response is qualitative with classes $k = 1, \ldots, K$
- The prediction for an observation is the most commonly occurring class among the training observations in its region
- Let $\hat{p}_{mk}$ be the proportion of training observations in region R_m that belong to class k

$$\hat{p}_{mk} = \frac{1}{|R_m|} \sum_{i:\, \mathbf{x}^{(i)} \in R_m} \mathrm{I}\left(y^{(i)} = k\right), \qquad \hat{y}_{R_m} = \arg\max_k \hat{p}_{mk}$$

- RSS cannot be used as the splitting criterion, so we need a measure of how *mixed* the classes are in a node

Node Impurity Measures

- **Classification error rate**: fraction of training observations in the region not in the majority class

$$E_m = 1 - \max_k \hat{p}_{mk}$$

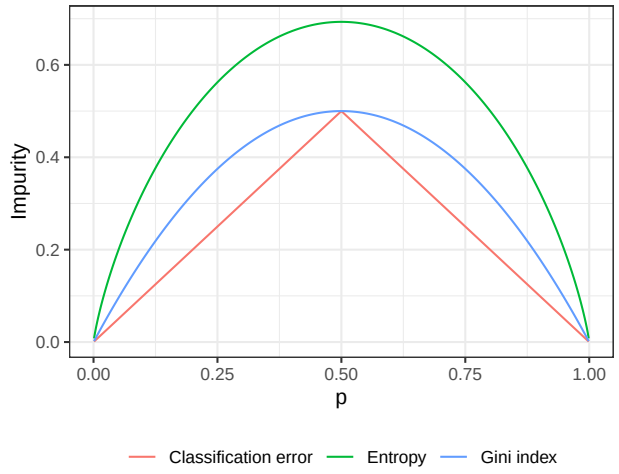
- **Gini index**: total variance across the K classes

$$G_m = \sum_{k=1}^{K} \hat{p}_{mk} \left(1 - \hat{p}_{mk}\right)$$

- **Entropy**

$$D_m = -\sum_{k=1}^{K} \hat{p}_{mk} \log \hat{p}_{mk}$$

Gini and Entropy as Measures of Node Purity



- Two classes, with p the proportion in class 1
- G_m and D_m are small when all $\hat{p}_{mk}$ are close to 0 or 1, so a small value indicates a **pure** node
- Gini and entropy are numerically very similar
- Both are differentiable and more sensitive to node purity than the error rate, which is piecewise linear in p

Choosing a Split: Worked Example

- Parent node with 400 observations in each of two classes; compare two candidate splits, each with (class 1, class 2) counts in the left and right child
- A split is scored by the size-weighted impurity of its children

$$\frac{n_L}{n_L + n_R} Q_L + \frac{n_R}{n_L + n_R} Q_R, \qquad Q \in \{E, G, D\}$$

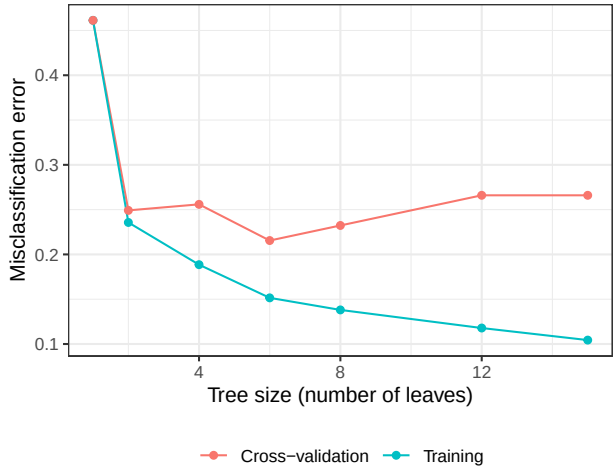
	Left	Right	Gini G	Entropy D
Split A	$(300, 100)$	$(100, 300)$	0.375	0.562
Split B	$(200, 400)$	$(200, 0)$	0.333	0.477

- Both splits have the same error rate, $E = 0.25$, but Gini and entropy prefer split B because it creates a pure node

Growing and Pruning Classification Trees

- When **growing** the tree, the Gini index or entropy is preferred, because they are more sensitive to node purity
- When **pruning**, any of the three can be used, but the classification error rate is preferable if the goal is prediction accuracy
- A split can leave both children with the same predicted class and still be worthwhile: it increases purity, hence our confidence in the prediction
- The pruning parameter α is chosen by cross-validation, exactly as for regression trees

Classification Tree: Heart Data



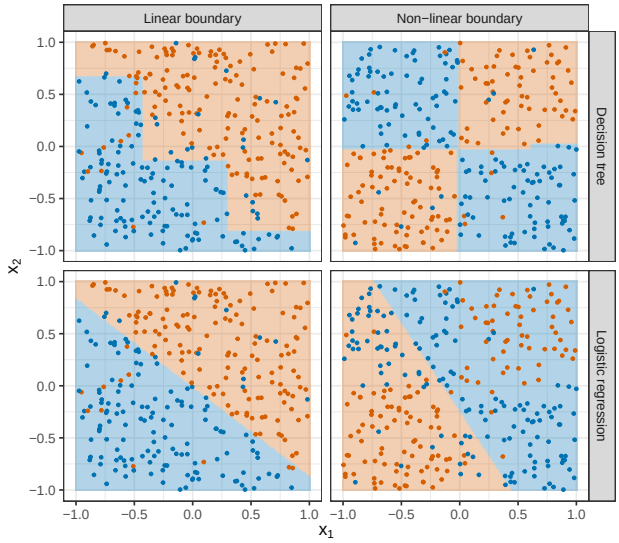
- Binary outcome: heart disease (`AHD`) `Yes`/`No`, with 13 predictors
- Training error keeps falling as the tree grows
- CV error flattens or rises beyond a small tree
- The pruned tree has comparable accuracy to the full tree and is much easier to read

Qualitative Predictors in Trees

- A split on a qualitative predictor assigns some of its levels to one branch and the rest to the other, e.g. `ChestPain` $\in \{\text{typical}, \text{nonanginal}\}$
- No dummy variables are needed
- A predictor with L unordered levels admits $2^{L-1} - 1$ distinct binary splits
- Splits on quantitative predictors only depend on the ordering of the values, so trees are unaffected by monotone transformations and need no scaling

Trees Versus Linear Models

- The better model depends on the shape of the true boundary
- Approximately linear boundary: linear model wins; highly non-linear, axis-aligned structure: tree wins



Advantages and Disadvantages of Trees

Advantages

- Very easy to explain
- Closely mirror human decision-making
- Displayed graphically, easy to interpret
- Handle qualitative predictors without dummy variables

Disadvantages

- Predictive accuracy is generally lower than other regression and classification methods
- Non-robust: a small change in the data can cause a large change in the final tree, i.e. high variance

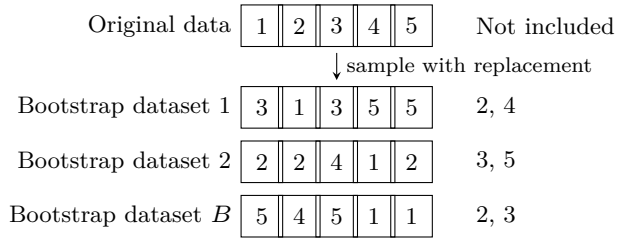
Aggregating many trees can substantially improve predictive performance.

4 Ensemble Methods: Bagging, Random Forests and Boosting

The Bootstrap: Background

- We have only one dataset, but want to know how an estimate (e.g. a mean or a tree's prediction) would change on other datasets
- The **bootstrap** creates new datasets by sampling n observations **with replacement** from the original n observations
- Some observations appear several times in a bootstrap dataset, others not at all
- Repeating this B times gives B bootstrap datasets, and the estimate computed on each shows how much it varies

The Bootstrap: Illustration



- Numbers are observation indices: each bootstrap dataset repeats some observations and omits others, e.g. dataset 1 contains observations 3 and 5 twice and omits 2 and 4
- Compute the estimate on each bootstrap dataset; the spread of these B estimates measures its uncertainty

Reducing Variance by Averaging

- Given independent random variables $Z_1, \ldots, Z_n$, each with variance σ^2, the mean has a smaller variance

$$\text{Var}(\bar{Z}) = \frac{\sigma^2}{n}$$

- Analogy: fit a separate tree on many training sets, then average the predictions to reduce variance
- We do not have many training sets, so we create pseudo-training sets using the **bootstrap**: sample n observations with replacement from the training data

Bagging

- **B**ootstrap **agg**regat**ing**: draw B bootstrap samples, fit a tree $\hat{f}^{(b)}$ to the b-th sample, and average

$$\hat{f}_{\text{bag}}(\mathbf{x}) = \frac{1}{B}\sum_{b=1}^{B} \hat{f}^{(b)}(\mathbf{x})$$

- Trees are grown deep and **not pruned**: each has high variance but low bias, and averaging reduces the variance
- For classification, take a **majority vote** over the B predicted classes
- Using a large B does not lead to overfitting; choose B large enough that the error has settled

Out-of-Bag Error Estimation

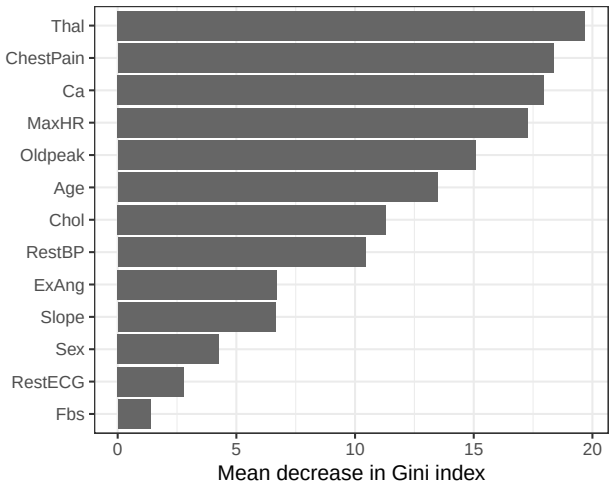
- The probability that observation i appears in a given bootstrap sample is

$$1 - \left(1 - \frac{1}{n}\right)^n \xrightarrow{n\to\infty} 1 - e^{-1} \approx 0.632$$

- So each tree uses about two-thirds of the observations; the remaining third are **out-of-bag** (OOB)
- Predict observation i using only the trees for which it was OOB (about $B/3$ trees), by averaging or majority vote
- The OOB MSE or classification error is a valid estimate of test error, since each prediction uses a model that never saw the observation
- For large B it is essentially equivalent to leave-one-out cross-validation, with no refitting

Variable Importance

- A bagged ensemble is no longer a single readable tree
- **Variable importance**: the total decrease in RSS (regression) or Gini index (classification) due to splits on a predictor, averaged over all B trees
- A large value indicates an important predictor



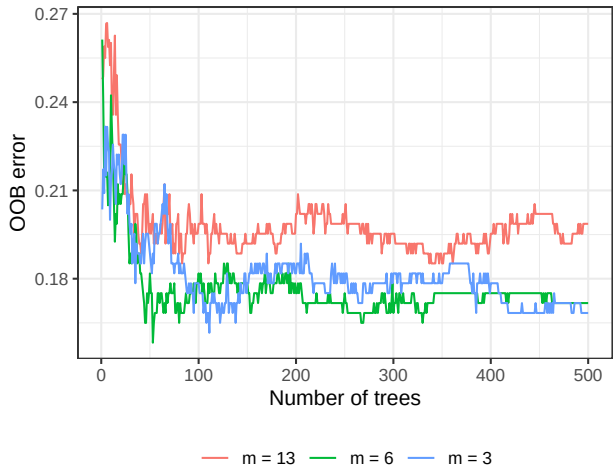
Random Forests

- If there is one very strong predictor, most bagged trees split on it first, so their predictions are highly correlated
- For B identically distributed variables with variance σ^2 and pairwise correlation ρ

$$\text{Var}\left(\frac{1}{B}\sum_{b=1}^{B} Z_b\right) = \rho\,\sigma^2 + \frac{1-\rho}{B}\,\sigma^2$$

- As $B \to \infty$ the variance does not go below $\rho\sigma^2$, so correlated trees limit the gain from averaging
- **Random forests decorrelate** the trees: at each split only a random subset of m of the p predictors is considered
- Typical choice: $m \approx \sqrt{p}$ for classification and $m \approx p/3$ for regression; $m = p$ is bagging

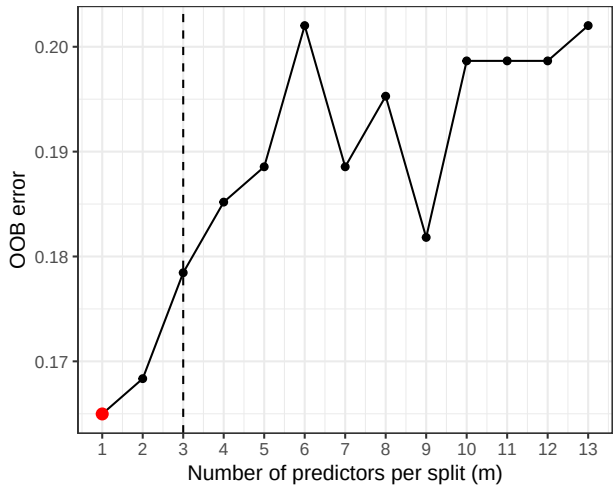
Bagging Versus Random Forests: Heart Data



- OOB error against the number of trees for $m = p$ (bagging), $p/2$ and $\sqrt{p}$
- Error stabilises after a modest number of trees
- Smaller m often helps, especially with many correlated predictors

Random Forests: Tuning Parameters

- **Number of trees** B: as in bagging, a larger B does not overfit; take it large enough that the error has settled
- **Predictors per split** m: the main tuning parameter; start from $\sqrt{p}$ (classification) or $p/3$ (regression) and try other values between 1 and p
- **Minimum node size**: trees are grown deep and not pruned, so this is usually left small



Choose m by the smallest OOB error, which needs no extra cross-validation.

Parallel Versus Sequential Ensembles

- **Bagging and random forests** grow trees in **parallel**, each on its own bootstrap sample, independently of the others
  - Individual trees are deep: low bias, high variance
  - Averaging reduces **variance**; bias stays about that of a single deep tree
- **Boosting** grows trees **sequentially**, each fit to the residuals of the trees so far
  - Individual trees are small: high bias, low variance
  - Adding trees mainly reduces **bias**; the shrinkage λ and the number of trees B control the variance

Boosting: Idea

- Like bagging, boosting is a general approach that combines many trees
- Trees are grown **sequentially**: each tree is fit using information from the previously grown trees
- No bootstrap sampling: each tree is fit to a modified version of the original data, namely the current **residuals**
- **Learn slowly**: each new small tree is added with a shrinkage factor λ, slowly improving $\hat{f}$ in areas where it does not perform well

Boosting Algorithm for Regression Trees

1. Set $\hat{f}(\mathbf{x}) = 0$ and $r^{(i)} = y^{(i)}$ for all i in the training set
2. For $b = 1, \ldots, B$, repeat:
   (a) Fit a tree $\hat{f}^{(b)}$ with d splits ($d + 1$ leaves) to the training data $(\mathbf{X}, \mathbf{r})$

(b) Update $\hat{f}$ by adding a shrunken version of the new tree

$$\hat{f}(\mathbf{x}) \leftarrow \hat{f}(\mathbf{x}) + \lambda \hat{f}^{(b)}(\mathbf{x})$$

(c) Update the residuals

$$r^{(i)} \leftarrow r^{(i)} - \lambda \hat{f}^{(b)}(\mathbf{x}^{(i)})$$

3. Output the boosted model

$$\hat{f}(\mathbf{x}) = \sum_{b=1}^{B} \lambda \hat{f}^{(b)}(\mathbf{x})$$

Why Fit the Residuals?

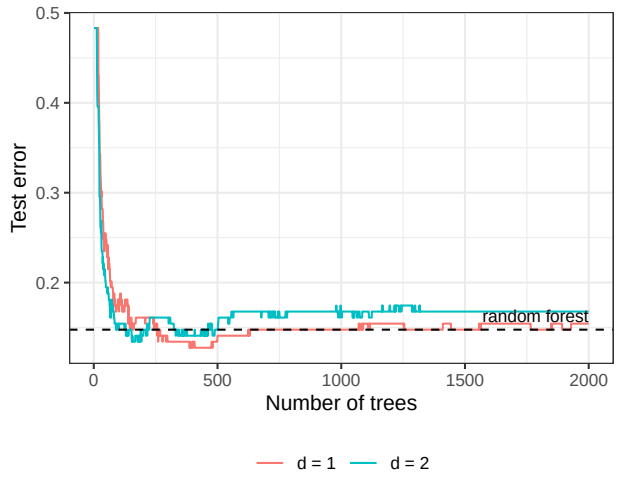
- For squared-error loss $L(y, f(\mathbf{x})) = \frac{1}{2}(y - f(\mathbf{x}))^2$, the negative gradient with respect to $f(\mathbf{x}^{(i)})$ is the residual

$$-\frac{\partial L}{\partial f(\mathbf{x}^{(i)})} = y^{(i)} - f(\mathbf{x}^{(i)}) = r^{(i)}$$

- Fitting a tree to the residuals and adding λ times it is a gradient descent step in function space, with step size λ
- Boosting is therefore a stagewise way of minimising the training loss
- For classification, the same idea is applied with a different loss, such as the binomial deviance (gradient boosting)

Boosting: Tuning Parameters

- **Number of trees** B: unlike bagging and random forests, too large a B can overfit; choose by cross-validation
- **Shrinkage** λ: typically 0.01 or 0.001; a small λ needs a large B
- **Depth** d: controls interaction order; $d = 1$ (stumps) gives an additive model $\sum_j f_j(x_j)$



Summary of Tree Ensembles

Method	Trees	Combination	Tuning	Reduces
Bagging	Deep trees on bootstrap samples	Average or majority vote	B	Variance
Random forest	As bagging, random m of p predictors per split	Average or majority vote	B, m	Variance
Boosting	Small trees grown sequentially on residuals	Sum of trees scaled by λ	B, λ, d	Bias

Key Takeaways

- Trees partition the predictor space into boxes and predict by the region mean or majority class; splits are chosen greedily by RSS, Gini or entropy
- Large trees overfit, so prune using cost complexity with α chosen by cross-validation
- Bagging reduces variance by averaging bootstrap trees, with OOB error as a free test error estimate
- Random forests decorrelate the trees by restricting the predictors considered at each split
- Boosting grows small trees sequentially on residuals